САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Лабораторная работа №3 «Контейнеризация написанного приложения средствами Docker»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать контейнеризацию микросервисного приложения фото-соцсети (созданного в ЛР2) средствами Docker:

- написать Dockerfile для каждого из четырёх сервисов;
- написать общий docker-compose.yml, описывающий все сервисы, RabbitMQ, сеть и volumes;
- настроить сетевое взаимодействие между контейнерами.

Ход работы

1. **Подготовка сервисов к контейнеризации**

Для корректной работы внутри контейнера пути к БД и папке uploads были вынесены в переменные окружения DB_PATH и UPLOADS_DIR. Это позволяет монтировать данные через Docker volumes и не привязываться к локальной структуре файловой системы.

Пример из services/auth-service/src/data-source.ts:

```
const dbPath = process.env.DB_PATH || path.join(__dirname, "../auth.sqlite");

export const AppDataSource = new DataSource({
  type: "sqlite",
  database: dbPath,
  synchronize: true,
  logging: false,
  entities: [User],
  migrations: [],
  subscribers: [],
});
```

Пример из services/auth-service/src/utils/imageProcessor.ts:

```
export const UPLOADS_DIR =
  process.env.UPLOADS_DIR || path.resolve(__dirname, "../../../../uploads");
```

2. **Dockerfile для каждого сервиса**

Каждый сервис имеет собственный Dockerfile, построенный по **двухстадийной схеме** (multi-stage build).

Стадия 1 (builder):

- базовый образ node:20-slim;
- установка системных зависимостей (python3, make, g++) для сборки нативных модулей sharp и sqlite3;
- установка npm-зависимостей (npm ci);

- компиляция TypeScript (npm run build).

Стадия 2 (runtime):

- тот же базовый образ;
- установка только production-зависимостей (npm ci --omit=dev);
- копирование скомпилированного dist/ из стадии builder;
- EXPOSE соответствующего порта;
- CMD ["node", "dist/index.js"].

Файл services/auth-service/Dockerfile

```
# ---------- Stage 1: build ----------
FROM node:20-slim AS builder

WORKDIR /app

# Зависимости для нативной сборки sharp / sqlite3
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 make g++ && rm -rf /var/lib/apt/lists/*

COPY package*.json ./
RUN npm ci

COPY tsconfig.json ./
COPY src ./src

RUN npm run build

# ---------- Stage 2: runtime ----------
FROM node:20-slim

WORKDIR /app

# Те же утилиты нужны для загрузки prebuilt-бинарей sharp/sqlite3
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 make g++ && rm -rf /var/lib/apt/lists/*

COPY package*.json ./
RUN npm ci --omit=dev

COPY --from=builder /app/dist ./dist

RUN mkdir -p temp-uploads

EXPOSE 3001
CMD ["node", "dist/index.js"]
```

Dockerfile для post-service, comment-service и social-service идентичны за исключением строки EXPOSE (3002, 3003 и 3004 соответственно).

Дополнительно в каждой папке сервиса создан файл .dockerignore, исключающий из контекста сборки node_modules, dist, *.sqlite, temp-uploads, .env, .vscode и *.log.

3. Общий docker-compose.yml

Файл services/docker-compose.yml описывает все сервисы, брокер сообщений, общую сеть и volumes.

```yaml
version: "3.9"

services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks:
      - backend

  auth-service:
    build:
      context: ./auth-service
      dockerfile: Dockerfile
    container_name: auth-service
    ports:
      - "3001:3001"
    environment:
      JWT_SECRET: your_super_secret_key_change_me
      SERVICE_TOKEN: internal_service_token_change_me
      DB_PATH: /app/data/auth.sqlite
      UPLOADS_DIR: /app/uploads
    volumes:
      - auth-db:/app/data
      - uploads-data:/app/uploads
    networks:
      - backend
    depends_on:
      - rabbitmq
```

```yaml
  post-service:
    build:
      context: ./post-service
      dockerfile: Dockerfile
    container_name: post-service
    ports:
      - "3002:3002"
    environment:
      JWT_SECRET: your_super_secret_key_change_me
      SERVICE_TOKEN: internal_service_token_change_me
      DB_PATH: /app/data/post.sqlite
      UPLOADS_DIR: /app/uploads
      AUTH_SERVICE_URL: http://auth-service:3001
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    volumes:
      - post-db:/app/data
      - uploads-data:/app/uploads
    networks:
      - backend
    depends_on:
      - rabbitmq
      - auth-service

  comment-service:
    build:
      context: ./comment-service
      dockerfile: Dockerfile
    container_name: comment-service
    ports:
      - "3003:3003"
    environment:
      JWT_SECRET: your_super_secret_key_change_me
      SERVICE_TOKEN: internal_service_token_change_me
      DB_PATH: /app/data/comment.sqlite
      UPLOADS_DIR: /app/uploads
      AUTH_SERVICE_URL: http://auth-service:3001
      POST_SERVICE_URL: http://post-service:3002
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    volumes:
      - comment-db:/app/data
      - uploads-data:/app/uploads
    networks:
      - backend
    depends_on:
      - rabbitmq
      - auth-service
      - post-service

  social-service:
    build:
      context: ./social-service
```

```
      dockerfile: Dockerfile
    container_name: social-service
    ports:
      - "3004:3004"
    environment:
      JWT_SECRET: your_super_secret_key_change_me
      SERVICE_TOKEN: internal_service_token_change_me
      DB_PATH: /app/data/social.sqlite
      AUTH_SERVICE_URL: http://auth-service:3001
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    volumes:
      - social-db:/app/data
    networks:
      - backend
    depends_on:
      - rabbitmq
      - auth-service

volumes:
  auth-db:
  post-db:
  comment-db:
  social-db:
  uploads-data:

networks:
  backend:
    driver: bridge
```

4. Сетевое взаимодействие между сервисами

Все контейнеры подключены к **единой пользовательской bridge-сети** backend. Внутри этой сети сервисы обращаются друг к другу **по имени контейнера**:

- Post Service → Auth Service: http://auth-service:3001/internal/users/...
- Comment Service → Auth Service: http://auth-service:3001/internal/users/...
- Comment Service → Post Service: http://post-service:3002/internal/posts/...
- Post / Comment Service → RabbitMQ: amqp://guest:guest@rabbitmq:5672

Наружу открыты только 4 порта (3001–3004) и порт RabbitMQ (5672, 15672). Все внутренние эндпоинты /internal/* защищены заголовком X-Service-Token и недоступны из внешнего мира напрямую.

5. Volumes

Используются **именованные volumes**, чтобы данные сохранялись между перезапусками контейнеров:

Volume	Что хранит	Куда монтируется
auth-db	База auth.sqlite	/app/data в auth-service
post-db	База post.sqlite	/app/data в post-service
comment-db	База comment.sqlite	/app/data в comment-service
social-db	База social.sqlite	/app/data в social-service
uploads-data	Аватары, посты, фото комментариев	/app/uploads в auth, post, comment

Volume uploads-data общий для трёх сервисов — это позволяет сервису Post отдавать те же файлы аватаров, что загружены через Auth.

6. Сборка и запуск

cd services

docker compose up –build

Docker собирает 4 образа (multi-stage), запускает 5 контейнеров (4 сервиса + RabbitMQ) в правильном порядке (с учётом depends_on).

Пример вывода после успешного старта:

rabbitmq | Server startup complete

auth-service | [auth-service] Database connected.

auth-service | [auth-service] Running on http://localhost:3001

post-service | [post-service] Database connected.

post-service | [post-service] RabbitMQ connected.

post-service | [post-service] Running on http://localhost:3002

comment-service | [comment-service] Database connected.

comment-service | [comment-service] RabbitMQ connected.

comment-service | [comment-service] Running on http://localhost:3003

social-service | [social-service] Database connected.

social-service | [social-service] RabbitMQ connected and consuming events.

social-service | [social-service] Running on http://localhost:3004

7. Проверка работоспособности

Health-check всех четырёх сервисов снаружи:

Invoke-RestMethod -Uri http://localhost:3001/ # -> auth-service ok

Invoke-RestMethod -Uri http://localhost:3002/ # -> post-service ok

Invoke-RestMethod -Uri http://localhost:3003/ # -> comment-service ok

Invoke-RestMethod -Uri http://localhost:3004/ # -> social-service ok

Проверка внутренней сети (из контейнера в контейнер):

docker exec -it post-service node -e "fetch('http://auth-service:3001/').then(r=>r.json()).then(console.log)"

Результат: { service: 'auth-service', status: 'ok' } — доказательство корректной работы сети.

8. Сквозное тестирование

Выполнен сценарий, аналогичный ЛР2, но все запросы идут против контейнеризованных сервисов:

1. **Регистрация** первого пользователя (id=1) через POST /auth/register.
2. **Логин** → JWT.
3. **Загрузка аватара** → avatarPath.
4. **Создание поста** с фото → postId=1.
5. **Лайк** собственного поста.
6. **Комментарий** к своему посту.
7. **Регистрация второго пользователя** (id=2, Alice).
8. **Логин Alice**.
9. **Alice лайкает пост Bogdan** → Post Service публикует like.created → Social Service создаёт уведомление.

Логи:

[post-service] Event published: like.created { userId: 2, postId: 1, postOwnerId: 1 }

[social-service] Received event: like.created { userId: 2, postId: 1, postOwnerId: 1 }

[social-service] Notification created for like: 1

10. **Alice комментирует** → аналогично уведомление comment.
11. **Notifications у Bogdan** → массив из двух уведомлений (like + comment).
12. **Mark as read** → isRead: true.
13. **Follow** → уведомление у Alice.
14. **Негативные проверки:**
    - Повторный лайк → Already liked (409).

- Подписка на себя → Cannot follow yourself (400).
- Комментарий к несуществующему посту → Post not found (404).

15. **Удаление поста** → Post Service публикует post.deleted → Social Service удаляет связанные уведомления.

Логи:

[post-service] Event published: post.deleted { postId: 1, userId: 1 }

[social-service] Received event: post.deleted { postId: 1, userId: 1 }

[social-service] Notifications removed for post: 1

Проверка: GET /social/notifications возвращает пустой массив.

9. Проверка изоляции сервисов (устойчивость к отказу)

Для демонстрации слабой связанности выполнен следующий эксперимент:

1. Остановлен Social Service: docker stop social-service.
2. Создан новый пост (id=2) и поставлен лайк от второго пользователя.
3. **Post Service продолжил работу**, принял запросы и опубликовал событие:

[post-service] Event published: like.created { userId: 2, postId: 2, postOwnerId: 1 }

4. Social Service запущен снова: docker start social-service.
5. Social Service «догнал» очередь и обработал накопившееся событие:

[social-service] Received event: like.created { userId: 2, postId: 2, postOwnerId: 1 }

[social-service] Notification created for like: 4

6. Проверка: GET /social/notifications показал новое уведомление по посту 2

Это доказывает:

- Post Service **не зависит** от Social Service и продолжает работать при его недоступности.
- Сообщения **не теряются** — они буферизуются в RabbitMQ благодаря durable: true и persistent: true.
- После перезапуска подписчик **гарантированно обрабатывает** все накопившиеся события.

Вывод

В ходе выполнения ЛР3 микросервисное приложение фото-соцсети полностью контейнеризировано:

- Для каждого из четырёх сервисов написан **multi-stage Dockerfile**, оптимизированный по размеру (в runtime-образ попадают только production-зависимости и скомпилированный код).
- Написан общий docker-compose.yml, описывающий 4 микросервиса, RabbitMQ, общую сеть и 5 volumes.
- Настроено **сетевое взаимодействие** между контейнерами: сервисы обращаются друг к другу по имени внутри bridge-сети backend, внешние /internal/* эндпоинты закрыты сервисным токеном.
- Данные (БД и загруженные файлы) сохраняются в **именованных volumes** и не теряются при перезапуске контейнеров.
- Проведено **сквозное тестирование** против контейнеризованного приложения — все сценарии (регистрация, посты, лайки, комментарии, подписки, уведомления, удаление) прошли успешно.
- Дополнительно проверена **устойчивость к отказам**: при остановке Social Service события буферизуются в RabbitMQ и обрабатываются после его перезапуска.

Полученная конфигурация полностью готова к развёртыванию на удалённом сервере (ЛР4) и может использоваться как основа для настройки CI/CD.